

Titre

Mini-Projet d'Algorithmique

Préparé par :

- Ourari Ranim
- Ayadi Soumaya



1. Introduction

La langue arabe repose sur un système morphologique racine–schème : chaque mot est formé en insérant une racine trilitère dans un schème abstrait. Notre solution consiste en un moteur morphologique permettant la gestion, la génération et la validation de mots arabes dérivés, s'appuyant sur un arbre AVL pour l'indexation des racines et une table de hachage pour la gestion des schèmes, tout en respectant des contraintes de complexité. En complément, nous avons développé une application mobile transformant le moteur en dictionnaire morphologique accessible.

2. Architecture du système

2.1 Architecture algorithmique

L'architecture algorithmique du système repose principalement sur un **arbre AVL** pour la gestion des racines arabes et une **table de hachage** pour la gestion des schèmes morphologiques. Les racines trilitères sont stockées dans l'arbre AVL afin de garantir une recherche et une insertion efficaces en complexité $O(\log n)$, grâce au mécanisme d'équilibrage par rotations. Les schèmes sont enregistrés dans une structure de type **HashMap**, permettant un accès rapide en complexité moyenne $O(1)$. La classe **Morphologie** constitue le noyau central du moteur et assure la coordination entre ces deux structures, en implémentant les opérations de **génération** et de **validation morphologique**. Le fonctionnement général suit les étapes suivantes : recherche de la racine dans l'arbre, accès au schème via la table de hachage, application de la règle de transformation, puis stockage du mot généré dans la liste des dérivés associée à la racine. L'initialisation des racines est réalisée **à partir d'un fichier texte chargé dynamiquement**, et un mécanisme d'adaptation d'affichage est utilisé afin de garantir une représentation correcte des caractères arabes dans le terminal.

2.2 Architecture logicielle

En complément du moteur algorithmique développé en **Java**, une architecture logicielle a été mise en place pour transformer le système en dictionnaire interactif. Le backend a été réalisé avec **Spring Boot** et les données sont stockées dans **MongoDB**. Une application mobile a été développée avec **React Native** en utilisant **TypeScript**. Le projet est structuré avec **Apache Maven** pour assurer une organisation claire et une gestion efficace des dépendances. Cette architecture représente une extension du moteur morphologique initial.

3. Choix des Structures de Données

Nous avons opté pour un **Arbre AVL** pour la gestion des racines. Ce choix est justifié par la nécessité de maintenir un dictionnaire ordonné tout en garantissant une complexité de recherche logarithmique $O(\log n)$, évitant ainsi la dégradation des performances propre aux arbres de recherche classiques non équilibrés. Pour les schèmes morphologiques, nous utilisons une **Table de Hachage** (HashMap). Cette structure est idéale car l'accès à un schème spécifique (comme "مفعول") se fait en temps constant $O(1)$, ce qui accélère considérablement le processus de génération et de validation morphologique. Enfin, chaque

nœud de l'arbre contient une **Liste Dynamique** pour stocker les mots dérivés validés, permettant une flexibilité totale selon la productivité de chaque racine.

4. Moteur de génération morphologique

Le moteur de génération morphologique constitue le cœur du système. À partir d'une racine trilitère et d'un schème sélectionné, le système génère automatiquement le mot dérivé correspondant. La première étape consiste à extraire les trois consonnes de la racine (**c1**, **c2**, **c3**). Ensuite, la règle de transformation associée au schème est appliquée : les lettres de la racine sont insérées dans les positions définies par le schème afin de produire le mot final. Une fois le mot généré, celui-ci est automatiquement ajouté à la liste des dérivés associés à la racine dans l'arbre AVL.

Un point fort du système réside dans la prise en charge des schèmes dynamiques. En effet, en plus des schèmes prédéfinis, le moteur permet de créer automatiquement de nouvelles règles de transformation à partir d'un modèle contenant les symboles « ف، ع، ل », ce qui rend le système **extensible et flexible**.

Du point de vue algorithmique, la complexité globale de la génération est dominée par la recherche de la racine dans l'arbre AVL, soit **$O(\log n)$** . L'accès au schème est réalisé en **$O(1)$** , et l'application de la règle de transformation s'effectue en temps constant. Ainsi, la génération morphologique reste **efficace et optimisée**.

5. Validation Morphologique

La validation morphologique vérifie si un mot donné appartient bien à une racine spécifique. Algorithmiquement, le moteur recherche d'abord la racine dans l'arbre AVL (**$O(\log n)$**). Si elle existe, il génère tous les mots possibles en appliquant les schèmes disponibles à cette racine. Si le mot à valider correspond à l'un d'eux, la réponse est positive. Sa complexité est donc de **$O(m)$** , où m est le nombre de schèmes, ce qui est **très rapide**.

6. Gestion des Dérivés

La gestion des dérivés est assurée au niveau de chaque nœud de l'arbre AVL. Chaque objet **Node** contient une **liste dynamique** permettant de stocker les mots dérivés associés à une racine donnée, ainsi que les schèmes correspondants. Cette structure permet d'organiser les dérivés sous forme de famille morphologique liée directement à leur racine. La liste utilisée étant dynamique, elle permet l'**ajout progressif** des mots générés ou validés sans limite prédéfinie. Lorsqu'un mot est généré ou confirmé morphologiquement, il est **automatiquement** ajouté à la liste des dérivés du nœud concerné, évitant ainsi les duplications.

Ce mécanisme garantit que le moteur conserve un **historique morphologique structuré** pour chaque racine, facilitant l'affichage, la consultation et l'analyse des familles lexicales.

7. Analyse de Complexité Globale

L'ajout et la recherche d'une racine s'effectuent en **$O(\log n)$** grâce à l'AVL. L'accès à un schème est en **$O(1)$** grâce à la **HashMap**. La génération reste en **$O(\log n)$** , la validation en

O(m). Le système maintient des performances logarithmiques pour les opérations les plus fréquentes, avec une dégradation acceptable uniquement pour l'analyse inversée.

8. Extension : dictionnaire morphologique complet

En complément du moteur algorithmique, notre solution a été transformée en un dictionnaire morphologique complet et interactif, accessible via une application mobile comprenant plusieurs interfaces : un **dictionnaire** pour rechercher une **racine** et afficher ses **mots dérivés**, une **page de génération** permettant de créer automatiquement des mots à partir d'une **racine** et d'un **schème**, une **analyse morphologique** pour obtenir le **radical** d'un **mot dérivé** ou tous les **dérivés d'un radical**, ainsi qu'une **interface de gestion des schèmes et racines** pour **ajouter, modifier et valider les mots**, accompagnée d'une **fonctionnalité de lecture audio** à visée pédagogique et interactive. Les **données** sont stockées de manière durable dans **MongoDB**, tandis que le **backend**, développé avec **Spring Boot** et **Java** et structuré selon **Apache Maven**, expose les fonctionnalités via une **API REST**, et l'**application mobile** est réalisée avec **React Native**. Cette extension illustre ainsi le passage d'un moteur algorithmique académique à une application complète, fonctionnelle et pédagogique.

9. Réflexions sur les Obstacles Rencontrés

-L'un des principaux défis a été la **gestion de l'affichage bidirectionnel (RTL)** dans les interfaces en ligne de commande. Le terminal inversant souvent l'ordre des caractères arabes, nous avons dû **concevoir un utilitaire de formatage spécifique** pour assurer la lisibilité sans altérer la donnée brute en mémoire.

-Un autre obstacle majeur a été la **conception d'un système de schèmes "dynamiques"**. Initialement, les transformations étaient codées en dur, mais pour offrir plus de flexibilité, nous avons développé **une logique de parsing** capable de transformer n'importe quelle chaîne contenant les lettres de base en **une règle algorithmique de production**.

-**L'implémentation des rotations AVL** a représenté une difficulté conceptuelle importante. Assurer la **correction des quatre cas de déséquilibre** et **mettre à jour les hauteurs** dans le bon ordre après chaque rotation a demandé une phase de débogage rigoureuse. Cela nous a appris à raisonner formellement sur des structures récursives avant d'implémenter.

-**Le chargement des racines depuis un fichier texte** a aussi posé problème, notamment pour accéder aux ressources depuis le **classpath Java** selon l'environnement d'exécution. Nous avons adopté **getResourceAsStream** pour garantir la portabilité du chargement.

-Enfin, **la synchronisation entre le moteur Java et l'API Spring Boot** a nécessité de définir des contrats d'interface clairs pour que l'application mobile consomme les fonctionnalités du moteur de manière cohérente, ce qui nous a fait prendre conscience de l'importance de découpler la logique métier de la couche de communication.

10. Conclusion

Ce projet a conçu un moteur de génération morphologique performant, utilisant un arbre AVL pour les racines et une table de hachage pour les schèmes. L'extension vers une application mobile et une base de données a transformé ce moteur en un dictionnaire morphologique réel, démontrant la solidité de l'architecture.